Woxsen University

School of Technology



A

PROJECT REPORT

on

CLOUD TASK PLACEMENT USING MST, SHORTEST PATHS AND DYNAMIC PROGRAMMING

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

K. Komal Prajvesh 25WU0102114
Joy Peswani 25WU0102110
Harshit Jain 25WU0102094
Abhigna Sai 25WU0102075

Faculty: Dr. Syed Javeed Pasha

ABSTRACT

This project studies how to place replicas and virtual machines (VMs) across a cloud network so that average request latency is as low as possible while no server exceeds its capacity. Servers are modeled as graph nodes and link latencies as edge weights. Three classical algorithmic tools are combined: a Kruskal minimum spanning tree (MST) forms a low-cost backbone, Dijkstra and Floyd-Warshall shortest paths route requests, and an exact dynamic program (DP) on the MST selects the replica set. A minimum-cost-flow assignment then enforces the capacity limits.

All algorithms were implemented in Python and evaluated on 30 random 50-server topologies (about 124 links each), with per-site demand and a capacity of 45 VM slots per replica. The MST keeps only 29.2% of the total link latency (273 ms against 939 ms summed over links) at an average path stretch of 1.53. The DP matched brute-force optimality on 200 of 200 random test trees. With K = 8 replicas and shortest-path routing, DP placement reached an average latency of 6.04 ms, compared with 6.21 ms for greedy and 11.35 ms for random placement, a 47% reduction over random. When requests are routed over the backbone itself, the DP is exact before capacity is applied and beat greedy on 29 of 30 topologies (7.06 ms against 7.68 ms). On 14-server instances where exhaustive search is possible, DP placement was on average within 4.2% of the true optimum and greedy within 9.8%.

The study shows that the three techniques fit together into a practical and fast placement pipeline: the MST decides which links to keep, shortest paths decide how traffic moves, and the DP decides where replicas go. The report also documents where the approach is weaker (the DP optimizes over MST distances, so its advantage shrinks when traffic can use non-backbone shortcuts) and proposes extensions for heterogeneous capacities and dynamic workloads.

Keywords: *Cloud Computing, Task Placement, Replica Placement, Minimum Spanning Tree, Shortest Paths, Dynamic Programming, k-Median, Latency Optimization*

TABLE OF CONTENTS

Abstract ii
List of Tables v
List of Figures vi

1. INTRODUCTION 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Objectives 2
1.5 Dataset Overview (Simulated Network Model) 2
1.6 Scope and Limitations 3
1.7 Organization of the Report 4

2. LITERATURE REVIEW / Technology review 5
2.1 Replica and VM Placement in Cloud Systems 5
2.2 Minimum Spanning Trees 5
2.3 Shortest-Path Algorithms 5
2.4 The k-Median Problem and Tree Metrics 5
2.5 Dynamic Programming 6
2.6 Capacitated Assignment as Min-Cost Flow 6
2.7 Summary of Techniques Used 6
2.8 Evaluation Metrics 6
2.9 Research Gap 7

3. METHODOLOGY 8
3.1 System Overview 8
3.2 Network and Workload Model 8
3.3 Backbone Design with MST 9
3.4 Routing with Shortest Paths 9
3.5 Placement Selection with Dynamic Programming 9
3.6 Capacity-Aware Assignment 10
3.7 Baseline Methods 10
3.8 Experimental Configuration 11
3.9 Evaluation Methodology 11
3.10 Implementation Details 11

4. RESULTS AND DISCUSSION 12
4.1 Backbone Quality 12
4.2 Correctness Validation 12
4.3 Runtime 13
4.4 Placement Quality 13

4.5 Effect of Capacity 15
4.6 Optimality Gap on Small Instances 15
4.7 Error Analysis 16
4.8 Comparison with Prior Work 16
4.9 Discussion 16
5. CONCLUSION AND FUTURE WORK 17
5.1 Summary of Contributions 17
5.2 Limitations 17
5.3 Future Work 17
5.4 Concluding Remarks 18
References 19
Appendix I 21
Appendix II 22

LIST OF TABLES

Table 1.1: Simulated Network and Workload Parameters 3
Table 2.1: Algorithms Used and Their Roles 6
Table 3.1: Experimental Configuration 11
Table 4.1: Backbone Statistics over 30 Topologies (N = 50) 12
Table 4.2: Correctness Validation Results 12
Table 4.3: Runtime against Number of Servers (single machine, indicative values) 13
Table 4.4: Average Request Latency at K = 8 (30 topologies, N = 50, C = 45) 14
Table 4.5: Comparison with Exhaustive Search (N = 14, K = 3, C = 35) 15

LIST OF FIGURES

Figure 1.1: Simulated 50-Server Network with MST Backbone and DP-Selected Replicas 3

Figure 3.1: Methodology Flowchart 8

Figure 4.1: Runtime against Number of Servers.....13

Figure 4.2: Average Request Latency against Number of Replicas (mean ± standard deviation over 30 topologies).....14

Figure 4.3: Comparison with Exhaustive Search.....15

1. INTRODUCTION

Cloud providers run services across many servers connected by links of very different latency. Where a replica or VM is placed decides how far each request must travel, so placement directly shapes user-perceived response time. This project builds and evaluates a complete placement pipeline that treats the cloud as a weighted graph and uses three classical algorithmic ideas: a minimum spanning tree (MST) for the backbone, shortest paths for routing, and dynamic programming (DP) for choosing where replicas go.

1.1 Background

A data center network can be viewed as a graph in which servers are nodes and link latencies are edge weights. Replica placement asks which nodes should host copies of a service so that the demand of every site is served from a nearby copy. The problem has been studied for web proxies and content replicas [1], [2] and, more recently, for VM placement inside data centers [3], [4].

Three sub-problems appear together in practice. First, a cheap and connected set of links is needed to carry replication and control traffic. Second, requests must be routed along low-latency paths. Third, a limited number of replicas must be placed so that average latency is minimal and no server is overloaded. The unconstrained version of the third problem is the classical k-median problem [5].

1.2 Motivation

The motivation for this project comes from practical and academic considerations:

- **User experience:** lower average latency means faster responses for users spread across sites.
- **Cost control:** a backbone that keeps only a fraction of all links lowers the cost of replication traffic and monitoring.
- **Capacity safety:** placement that ignores server limits produces overloaded replicas, so capacity must be part of the model.
- **Learning value:** the problem brings together graph algorithms (MST, shortest paths) and algorithm design (DP, flows) in one realistic setting.

1.3 Problem Statement

Given a connected graph G = (V, E) with link latencies w(e) > 0, a demand d(i) at every site i (the number of VM requests it generates), a capacity C for each replica and a budget of K replicas, choose a replica set S of size K and an assignment of demand to replicas that minimizes the demand-weighted average latency. Here l(i, s) is the shortest-path latency between site i and replica s, x(i, s) is the share of site i's demand served by s, and D is the total demand.

$$\min_{S,\ x} \quad \frac{1}{D} \sum_{i \in V} \sum_{s \in S} \ell(i, s)\, x_{is}$$

$$\text{subject to} \quad \sum_{s \in S} x_{is} = d_i \quad \forall i \in V, \qquad \sum_{i \in V} x_{is} \leq C \quad \forall s \in S, \qquad x_{is} \geq 0, \qquad |S| = K$$

The main difficulties are:

- Choosing S is a combinatorial problem; exhaustive search needs C(N, K) evaluations.
- Capacity couples the sites together, because one site's choice of replica affects what is left for others.
- Latency depends on the routing regime: traffic may be forced onto the backbone or allowed to use any physical shortcut.
- Real networks are irregular, so simple geometric rules do not give good placements.

1.4 Objectives

The specific objectives of this project are:

1. To build a low-cost backbone of a latency-weighted server graph using a minimum spanning tree.
2. To compute routing latencies using shortest-path algorithms (Dijkstra and Floyd-Warshall) and compare backbone routing with shortest-path routing.
3. To select replica locations using an exact dynamic program on the MST and enforce server capacity with a minimum-cost-flow assignment.
4. To evaluate the pipeline against greedy, random and exhaustive-search placement using average latency, optimality gap, capacity utilization and runtime.

1.5 Dataset Overview (Simulated Network Model)

No public dataset gives server-to-server latencies together with per-site demand and capacity, so the study uses a controlled simulation. Servers are placed uniformly at random in a 100 × 100 area, each is linked to its four nearest neighbours, and any disconnected parts are joined by their closest pair of nodes. Table 1.1 lists the parameters and Figure 1.1 shows one generated topology.

Parameter	Value
Servers (nodes) N	50 (14 for exhaustive-search tests)
Physical links	About 124 per topology (4-nearest-neighbour graph plus connectivity repairs)
Link latency	(1 + 0.4 × distance) × U(1.0, 1.3) ms
Demand per site	Integer from 1 to 10 VM requests (mean 5.5, total about 275)
Capacity per replica	45 VM slots (35 for the small instances)
Replica budget K	8 to 14 (the smallest feasible K is 7)
Random topologies	30 for the main study, 20 for the optimality study

Table 1.1: Simulated Network and Workload Parameters

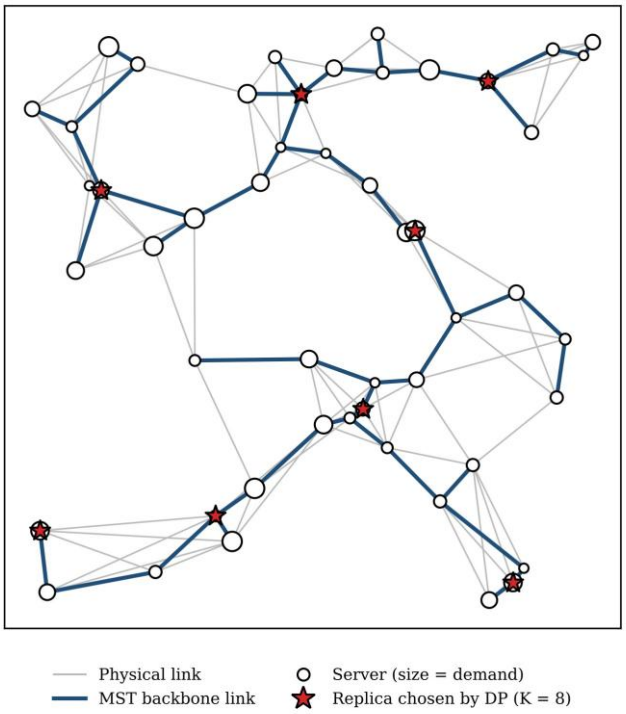


Figure 1.1: Simulated 50-Server Network with MST Backbone and DP-Selected Replicas

1.6 Scope and Limitations

The scope covers static placement on a single graph with uniform replica capacity and splittable demand. The limitations are:

- The topologies are synthetic, so absolute latencies will differ from a real provider network.

- Capacity is the same for every replica, and demand can be split across replicas (a linear-programming relaxation of VM-level assignment).
- Placement is computed once for a fixed demand pattern; online re-placement is not studied.
- Failures, bandwidth limits and monetary cost are not modeled.

1.7 Organization of the Report

Chapter 2 reviews work on replica placement, spanning trees, shortest paths, the k-median problem and dynamic programming. Chapter 3 describes the network model, the algorithms and the experimental setup. Chapter 4 presents results, including correctness checks, runtime, placement quality and optimality gaps. Chapter 5 concludes with contributions, limitations and future work.

2. LITERATURE REVIEW / Technology review

This chapter reviews the research and algorithms behind the three parts of the pipeline: backbone design, routing and placement selection.

2.1 Replica and VM Placement in Cloud Systems

Early work on content delivery modeled proxy placement as a median problem on the network. Li et al. formulated the placement of web proxies as a p-median problem on a tree and gave a polynomial-time dynamic program for it [1]. Qiu et al. compared several replica placement strategies on Internet-like topologies and found that a simple greedy strategy performs close to the optimum and clearly better than random or hot-spot placement [2].

In data centers the focus moved to VM placement. Meng et al. showed that placing VMs with awareness of their traffic reduces load on the network core [3]. Beloglazov et al. proposed energy-aware allocation heuristics that pack VMs onto fewer hosts while respecting service levels [4]. These studies treat placement, backbone design and routing mostly in isolation.

2.2 Minimum Spanning Trees

An MST connects all nodes of a weighted graph with the smallest total edge weight. Kruskal's algorithm sorts edges and adds each one that does not close a cycle, using a union-find structure [6]. Prim's algorithm grows a tree from a start node by repeatedly adding the cheapest edge leaving it [7]. Both run in O(E log V) time. For a latency-weighted network the MST is a natural low-cost backbone, with the trade-off that paths inside the tree can be longer than the best physical paths.

2.3 Shortest-Path Algorithms

Dijkstra's algorithm finds shortest paths from one source to all nodes in graphs with non-negative weights, in O((V + E) log V) time with a binary heap [8]. Floyd and Warshall's algorithm computes all-pairs shortest paths in $O(V^3)$ time by allowing one more intermediate node at each step [9], [10]. Running Dijkstra from every node gives the same answer and is faster on sparse graphs.

2.4 The k-Median Problem and Tree Metrics

Hakimi introduced the median problem on networks and showed that medians can be restricted to nodes [5]. Choosing K replicas to minimize total weighted distance is the k-median problem, which is NP-hard on general graphs. Constant-factor approximations exist [11], and local search with swaps gives a (3 + epsilon)-approximation [12]. On trees, however, the problem is solvable exactly: Kariv and Hakimi gave a polynomial algorithm [13], and Tamir improved the running time to $O(K N^2)$ [14]. This is the reason the project places replicas on the MST.

2.5 Dynamic Programming

Dynamic programming solves a problem by combining optimal solutions of overlapping subproblems [15]. Tree DPs process nodes from the leaves to the root and combine children with a knapsack-style merge over the number of replicas used in each subtree. The same table answers the question for every replica budget at once, which makes it easy to study how latency falls as K grows.

2.6 Capacitated Assignment as Min-Cost Flow

Once replicas are fixed, assigning demand to replicas under capacity limits is a transportation problem, a special case of minimum-cost flow that is solvable in polynomial time [16]. Sources are sites with supply d(i), sinks are replicas with capacity C, and arc costs are shortest-path latencies. This project solves it as a linear program.

2.7 Summary of Techniques Used

Technique	Role in this project	Time complexity	Reference
Kruskal MST	Backbone design	O(E log E)	[6]
Prim MST	Cross-check of the backbone	O(E log V)	[7]
Dijkstra	Single-source routing	O((V + E) log V)	[8]
Floyd-Warshall	All-pairs routing	$O(V^3)$	[9], [10]
Tree DP	Replica selection on the MST	$O(N^2 K^2)$ here; $O(K N^2)$ in [14]	[13], [14]
Min-cost flow (LP)	Capacity-aware assignment	Polynomial	[16]

Table 2.1: Algorithms Used and Their Roles

2.8 Evaluation Metrics

Placement quality is measured by the demand-weighted average latency, defined as total demand-times-latency divided by total demand. Other useful measures are the optimality gap against exhaustive search, the share of topologies where one method beats another, backbone cost relative to the full network, path stretch (average latency inside the backbone divided by average latency in the full network) and runtime.

2.9 Research Gap

Prior work optimizes one stage at a time. This project combines backbone design, routing and placement in one pipeline and measures a question that is rarely stated directly: how much does it cost to choose replicas using MST distances when real traffic may take non-backbone shortcuts? It also handles capacity explicitly through a flow assignment and reports exhaustive-search optimality gaps on small instances.

3. METHODOLOGY

This chapter describes the network model, the four stages of the pipeline, the baselines and the experimental setup.

3.1 System Overview

The proposed system has four stages. Stage 1 builds the backbone with Kruskal's MST. Stage 2 computes routing latencies with Dijkstra and Floyd-Warshall. Stage 3 selects K replica sites with an exact tree DP on the MST. Stage 4 assigns demand to the chosen replicas under capacity with a minimum-cost-flow model and reports the average latency. Figure 3.1 shows the flow.

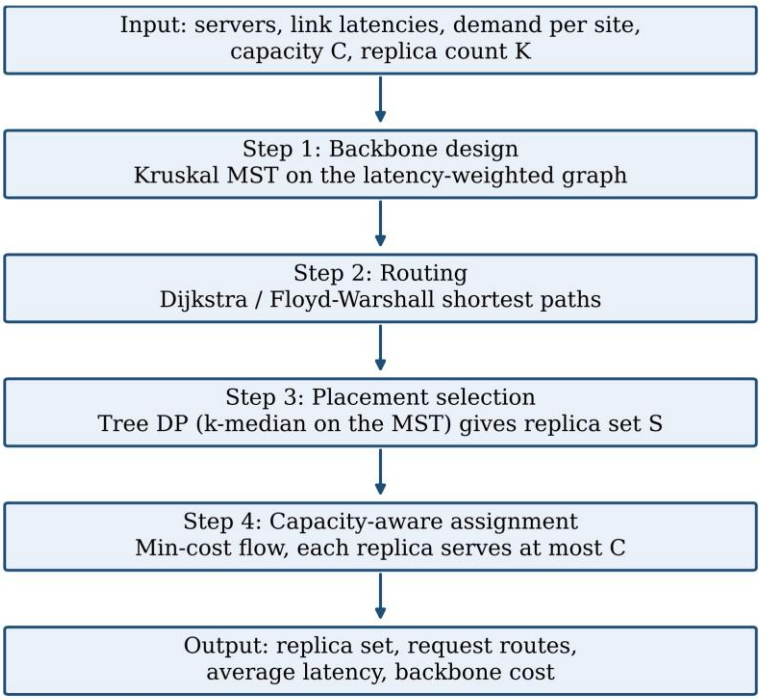


Figure 3.1: Methodology Flowchart

3.2 Network and Workload Model

3.2.1 Graph Model

The cloud is an undirected connected graph G = (V, E) with N servers and link latencies w(e) in milliseconds. Every server can host replicas and also generates demand d(i), the number of VM requests per time unit.

3.2.2 Topology Generation

Servers are drawn uniformly in a 100 × 100 area. Each server is linked to its four nearest neighbours, and any remaining components are joined through their closest pair of servers. Link latency is (1 + 0.4 × distance) × U(1.0, 1.3) ms, which gives a fixed base delay, a distance term and 0 to 30% random variation. Topologies are generated from seeds 0 to 29, so every experiment is reproducible.

3.2.3 Demand and Capacity

Demand at each site is an integer drawn uniformly from 1 to 10, so total demand D is about 275. Every replica has capacity C = 45, so at least ceil(D / C) = 7 replicas are needed and K = 8 corresponds to roughly 76% average utilization.

3.3 Backbone Design with MST

Kruskal's algorithm sorts the links by latency and keeps each link that joins two different components, tracked with a union-find structure with path compression and union by rank. The result is a tree with N - 1 links. Prim's algorithm is implemented separately and used only to confirm that both produce the same total weight. Two backbone measures are reported: the total latency of kept links against the full network, and the path stretch, which is the average pairwise path latency inside the tree divided by the same quantity in the full graph.

3.4 Routing with Shortest Paths

Dijkstra's algorithm with a binary heap is run from every server to obtain all-pairs latencies, and Floyd-Warshall is implemented with vectorized NumPy updates as an independent check. Two routing regimes are compared:

- **Backbone routing:** every request must travel along MST paths, so the distance between two servers is their tree distance.
- **Shortest-path routing:** requests may use any physical link, so the distance is the shortest-path latency in the full graph.

3.5 Placement Selection with Dynamic Programming

3.5.1 Why a Tree DP

The k-median problem is NP-hard on general graphs but exactly solvable on trees [13], [14]. The MST is a tree, so the DP gives the provably best uncapacitated replica set under backbone routing, and a strong heuristic under shortest-path routing, where tree distances overestimate real latency.

3.5.2 States and Recurrence

Root the MST at node 0. For a node v with subtree T(v), let f(v, j, u) be the minimum cost of serving all clients in T(v) with exactly j replicas inside T(v), when v itself is served by replica u. If u lies inside T(v) it must be open and is counted in j; if u lies outside, it is an existing replica that is not counted. Define the helper values below, where c ranges over the children of v.

$$m_c(j) = \min_{z \in T(c)} f(c, j, z)$$

$$X_c(j, u) = f(c, j, u) \qquad (u \in T(c))$$

$$X_c(j, u) = \min \{ f(c, j, u),\ m_c(j) \} \qquad (u \notin T(c))$$

$$f(v, j, v) = \min_{\sum_c j_c = j - 1} \sum_c X_c(j_c, v) \qquad (v \text{ is open})$$

$$f(v, j, u) = d_v \operatorname{dist}(v, u) + \min_{\sum_c j_c = j} \sum_c X_c(j_c, u) \qquad (u \neq v)$$

The answer for K replicas is $m_{root}(K)$. The key idea is that in a tree, a child is served either by the same replica as its parent or by a replica inside its own subtree, so only these two choices need to be considered.

3.5.3 Merging Children and Recovering the Replica Set

Children are merged one at a time with a min-plus convolution over the replica counts, which is the knapsack step of the DP. The merge is vectorized over all candidate servers u with NumPy. The index chosen at each merge is stored, and a backtracking pass from the root recovers the actual replica set.

3.5.4 Complexity

Each of the N nodes holds a table of (K + 1) × N entries, and each child merge costs $O(K^2)$ per column, so the total time is $O(N^2 K^2)$ and memory is O(N K) per stored table [17]. Tamir's algorithm reaches $O(K N^2)$ [14]; the simpler version is sufficient for the sizes studied here.

3.6 Capacity-Aware Assignment

The DP ignores capacity, so the chosen set is passed to a transportation model that sends each site's demand to replicas at minimum total latency, with each replica serving at most C. It is solved as a linear program with the HiGHS solver. If K × C is below the total demand the set is infeasible; the K values used here always satisfy this.

3.7 Baseline Methods

- **Greedy:** add replicas one at a time, each time choosing the server that lowers weighted distance the most, using the distance matrix of the routing regime being tested.
- **Random:** choose K servers uniformly at random and average over 5 draws per topology.
- **Exhaustive search:** on 14-server instances, evaluate every set of 3 replicas with the capacity-aware assignment to find the true optimum.

All methods are evaluated with the same capacity-aware assignment, so differences come only from the replica sets.

3.8 Experimental Configuration

Setting	Value
Main study	30 topologies, N = 50, C = 45, K = 8 to 14
Routing regimes	Backbone (tree distance) and shortest-path (full graph)
Optimality study	20 topologies, N = 14, K = 3, C = 35, exhaustive search
DP correctness test	200 random 11-node trees, K = 1 to 4, brute-force comparison
Runtime study	N = 50, 100, 200, 400; best of 3 runs (1 run for the $O(N^3)$ and all-source cases)
Random baseline	Mean of 5 random sets per topology and K

Table 3.1: Experimental Configuration

3.9 Evaluation Methodology

The main metric is the demand-weighted average latency in milliseconds after capacity-aware assignment. Results are reported as mean ± standard deviation over topologies. Additional measures are the optimality gap against exhaustive search, the fraction of topologies where DP is at least as good as greedy, the capacity penalty (capacity-aware latency against latency with capacity ignored), backbone cost savings, path stretch and runtime.

3.10 Implementation Details

Everything was implemented in Python using NumPy 2.5.3, SciPy 1.18.1 (HiGHS linear-programming solver) and Matplotlib 3.11.2. The MST, shortest-path and DP algorithms were written from scratch; only the linear-programming step uses a library. Experiments ran on a 2-vCPU cloud machine without a GPU. The code is available at [repository URL].

4. RESULTS AND DISCUSSION

This chapter reports the experimental results: backbone quality, correctness checks, runtime, placement quality, the effect of capacity and the optimality gap. All values are means ± standard deviations over random topologies unless stated otherwise.

4.1 Backbone Quality

The MST keeps 49 of about 124 links and only 29% of the total link latency, a saving of 70.8% ± 1.2%. The price is longer paths: the average pairwise latency inside the tree is 54.3 ms against 35.6 ms in the full network, a stretch of 1.53 ± 0.16. The backbone is therefore a good choice for replication and control traffic, but forcing user requests onto it would cost about 53% more latency on average.

Metric	**Full network**	**MST backbone**
Links	123.7 ± 3.2	49
Total link latency (ms)	938.7 ± 44.6	273.5 ± 11.0
Average pairwise path latency (ms)	35.6 ± 3.6	54.3 ± 5.3

Table 4.1: Backbone Statistics over 30 Topologies (N = 50)

4.2 Correctness Validation

Before comparing placements, each algorithm was checked against an independent method. All checks passed, including exact agreement of the DP with brute force on every test tree.

Check	**Result**
Dijkstra against Floyd-Warshall (all pairs, 30 topologies)	Maximum difference 4.3×10^{-14} ms
Kruskal against Prim (total MST weight)	Equal on 30 of 30 topologies
Tree DP against brute force (200 random 11-node trees, K = 1 to 4)	Exact on 200 of 200

Table 4.2: Correctness Validation Results

The DP check mattered in practice. A first version that charged a non-replica site only for the distance to an outside replica was exact on just 43 of 200 trees; allowing a site to use the nearest replica inside its own subtree, as in Section 3.5.2, fixed this.

4.3 Runtime

Table 4.3 gives the runtime of each algorithm as N grows and Figure 4.1 plots it on log axes. The MST algorithms stay below 1 ms even at 400 servers. All-sources Dijkstra and Floyd-Warshall dominate the cost at large N, and the tree DP with K = 10 takes about 167 ms at N = 400, which is acceptable for an offline planning step.

N	Kruskal (ms)	Prim (ms)	Dijkstra, all sources (ms)	Floyd-Warshall (ms)	Tree DP, K = 10 (ms)
50	0.07	0.07	2.45	0.35	6.68
100	0.16	0.14	9.42	1.69	14.90
200	0.31	0.31	38.65	11.73	39.06
400	0.76	0.75	242.33	184.01	166.90

Table 4.3: Runtime against Number of Servers (single machine, indicative values)

The DP time grows about 25 times when N grows 8 times, which is below the $O(N^2)$ bound for fixed K because NumPy overhead dominates at small N. Floyd-Warshall beats the pure-Python Dijkstra at small N because its inner loop is vectorized; the order reverses in theory on large sparse graphs.

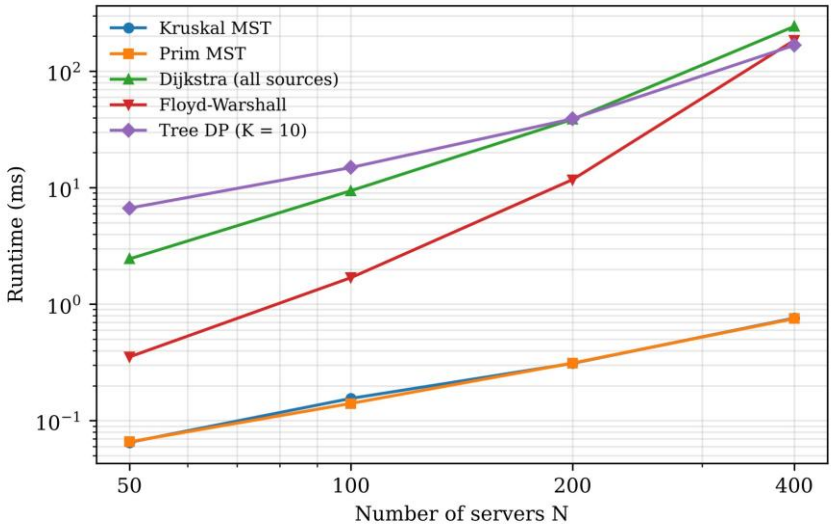


Figure 4.1: Runtime against Number of Servers

4.4 Placement Quality

Table 4.4 compares the methods at K = 8 for both routing regimes after capacity-aware assignment. The DP on the MST gives the lowest average latency in both regimes. Against random placement it lowers latency by 58.0% under backbone routing and by 46.8% under shortest-path routing.

Method	Backbone routing (ms)	Shortest-path routing (ms)
DP on MST	**7.06 ± 0.57**	**6.04 ± 0.46**
Greedy	7.68 ± 0.92	6.21 ± 0.40
Random (mean of 5 draws)	16.82 ± 2.91	11.35 ± 1.55
DP set, capacity ignored	6.80 ± 0.40	5.90 ± 0.40

Table 4.4: Average Request Latency at K = 8 (30 topologies, N = 50, C = 45)

Under backbone routing the DP is exact when capacity is ignored, and it beat greedy on 29 of 30 topologies; greedy was on average 8.9% worse (standard deviation 11.1%). Under shortest-path routing the picture is closer: the DP was at least as good as greedy on 20 of 30 topologies, and its average advantage was only 2.7% with a standard deviation of 6.1%. That difference is small relative to its spread, so the honest conclusion is that the DP is slightly better on average, not uniformly better.

Figure 4.2 extends the comparison to K = 8 through 14. The DP curve lies below the greedy curve at every K in both regimes, and both are far below random. Latency shows diminishing returns: under shortest-path routing the first extra replica (K = 8 to 9) saves 0.68 ms while the last (K = 13 to 14) saves 0.32 ms, and the total drop from K = 8 to 14 is 45%.

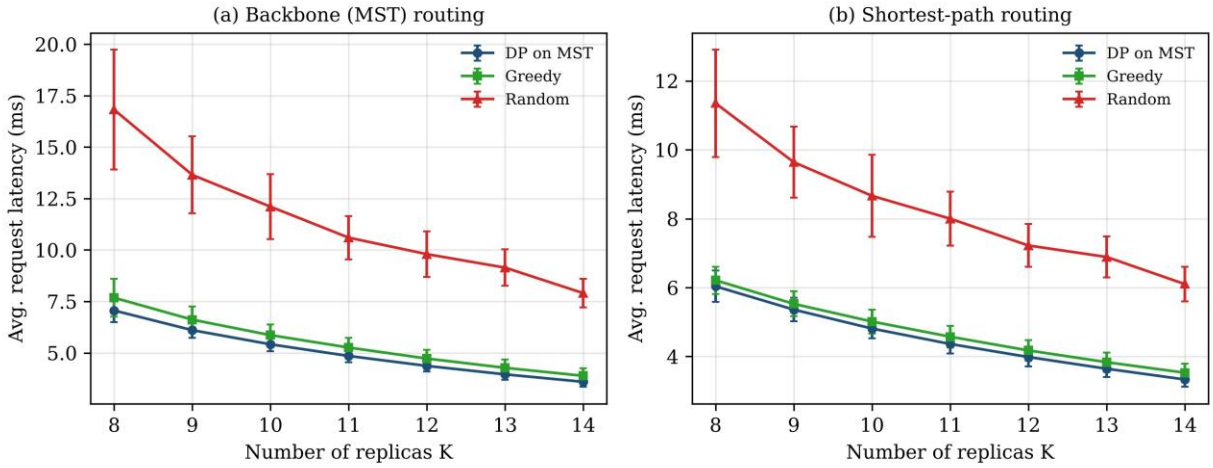


Figure 4.2: Average Request Latency against Number of Replicas (mean ± standard deviation over 30 topologies)

4.5 Effect of Capacity

Capacity matters most when replicas are close to full. At K = 8 the average utilization is about 76%, and enforcing capacity raises the latency of the DP set from 6.80 to 7.06 ms under backbone routing (3.8%) and from 5.90 to 6.04 ms under shortest-path routing (2.4%). At K = 14 utilization falls to about 44% and the penalty is zero. The DP therefore needs no capacity term when K is comfortably above the minimum, and a simple flow assignment is enough to repair the set when K is close to it.

4.6 Optimality Gap on Small Instances

To measure how far each method is from the best possible placement, all sets of 3 replicas were evaluated by exhaustive search on 20 topologies of 14 servers (shortest-path routing, capacity 35). The DP set was on average 4.2% above the optimum and the greedy set 9.8% above, while a random feasible set was 72.5% above. The DP found the exact optimum on 5 of 20 instances and greedy on 4 of 20. Figure 4.3 compares the average latencies.

Method	Average latency (ms)	Gap to optimum	Instances optimal
Exhaustive optimum	8.74 ± 0.84	0%	20 of 20
DP on MST	9.10 ± 0.84	4.2% ± 4.9%	5 of 20
Greedy	9.62 ± 1.34	9.8% ± 9.6%	4 of 20
Random (expected value)	15.00 ± 1.18	72.5% ± 16.1%	not applicable

Table 4.5: Comparison with Exhaustive Search (N = 14, K = 3, C = 35)

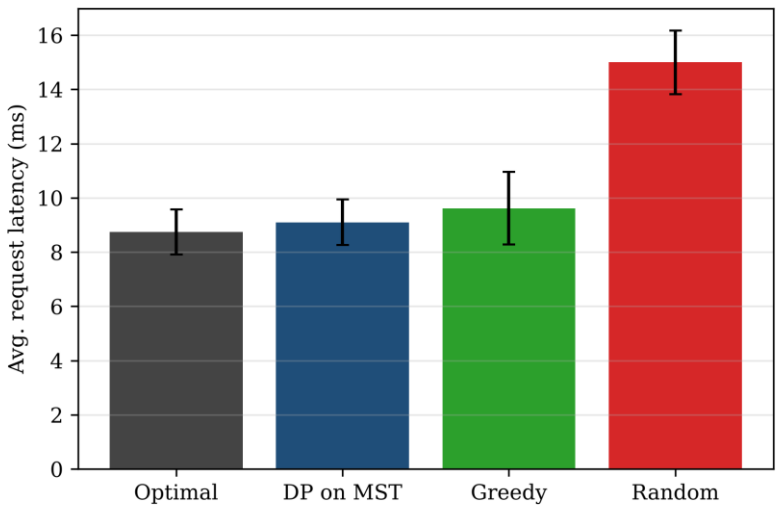


Figure 4.3: Comparison with Exhaustive Search

4.7 Error Analysis

The measurements point to two sources of loss in the DP. The first is the metric mismatch: the DP optimizes over tree distances, which are 1.53 times longer than shortest-path distances on average, so under shortest-path routing it can prefer a site that is good only inside the backbone. This explains why its gap to the optimum (4.2%) and its margin over greedy (2.7%) are small there, and why it is exact in the backbone regime. The second is capacity: the DP ignores it, which adds up to 3.8% at 76% utilization. Greedy loses for a different reason, since it adds one replica at a time and cannot undo early choices, which shows in its larger spread (9.6% standard deviation on the small instances).

4.8 Comparison with Prior Work

The results agree with earlier findings in three ways. Greedy placement is clearly better than random and reasonably close to the optimum, as reported in [2]; here it is 9.8% above the optimum on the small instances. A tree DP gives exact optima on trees, as in [1], [13], [14], and was confirmed on 200 of 200 test trees. And capacity-aware assignment as a flow problem is practical at these sizes [16]. A direct numeric comparison with the cited papers is not possible, because they use different topologies and workloads.

4.9 Discussion

First, the three techniques divide the work cleanly. The MST decides which links to keep and cuts link latency by about 71%, shortest paths decide how traffic moves, and the DP decides where replicas go. Second, the choice of routing regime matters as much as the algorithm: backbone routing raises average latency by about 17% at K = 8 (7.06 against 6.04 ms for the DP) compared with shortest-path routing. Third, the MST should be seen as a control and replication backbone, not as the only data path. Fourth, results come from synthetic topologies, so the absolute values are illustrative; the relative ordering of methods is the reliable finding.

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

This project built and evaluated a complete cloud task placement pipeline on a latency-weighted server graph. The main contributions are:

1. A backbone design with Kruskal's MST that keeps 29% of the link latency of the full network (273 ms against 939 ms) at a path stretch of 1.53.
2. A comparison of backbone routing and shortest-path routing using Dijkstra and Floyd-Warshall, which shows that forcing traffic onto the backbone raises average latency by about 17% at K = 8.
3. An exact tree DP for replica selection, verified against brute force on 200 of 200 random trees, combined with a minimum-cost-flow assignment that enforces capacity.
4. An evaluation on 30 random topologies showing 6.04 ms average latency for the DP against 6.21 ms for greedy and 11.35 ms for random placement (K = 8, shortest-path routing), and a 4.2% average gap to the optimum on 14-server instances.

5.2 Limitations

- The topologies and workloads are synthetic, so absolute latencies are illustrative.
- All replicas have the same capacity and demand may be split across replicas.
- The DP ignores capacity, so the flow step can only repair, not optimize, the chosen set.
- Under shortest-path routing the DP works on tree distances, so it is a heuristic there and its average advantage over greedy (2.7%) is small relative to its variation.
- The optimality study is limited to 14 servers and 3 replicas because it uses exhaustive search.

5.3 Future Work

5.3.1 Capacity Inside the DP

Adding a capacity dimension to the DP state, at least for uniform capacities, would let the recurrence choose sets that are good after assignment rather than before it.

5.3.2 Local-Search Refinement

Starting from the DP set and applying swap moves evaluated on the true shortest-path distances and the flow cost could close much of the remaining gap to the optimum [14].

5.3.3 Richer Backbones

Adding a few shortcut links to the MST, or using a k-edge-connected backbone, would reduce stretch and give fault tolerance at modest extra link cost.

5.3.4 Dynamic Workloads

Demand changes over the day. An online version could re-run the DP periodically and move replicas only when the latency gain outweighs migration cost.

5.3.5 Real Latency Data

The pipeline could be validated on measured inter-region or inter-data-center latencies and real request traces instead of the simulated network.

5.3.6 Multi-Objective Placement

Energy use, monetary cost and failure risk could be added to the objective, for example by extending the energy-aware heuristics in [4].

5.4 Concluding Remarks

MST, shortest paths and DP each solve a different part of cloud task placement, and together they give a fast and verifiable pipeline. The backbone cuts link cost by about 71%, the DP is exact on trees and beats greedy and random placement on average, and capacity can be handled with a flow step at little cost when replicas are not close to full. The main caveat is that the DP optimizes over backbone distances, so the largest gains appear when traffic is routed over the backbone, and the method is best used as a strong starting point for further refinement.

REFERENCES

[1] B. Li, M. J. Golin, G. F. Italiano, X. Deng and K. Sohraby, "On the optimal placement of web proxies in the Internet," in *Proc. IEEE INFOCOM*, 1999, pp. 1282-1290.

[2] L. Qiu, V. N. Padmanabhan and G. M. Voelker, "On the placement of web server replicas," in *Proc. IEEE INFOCOM*, 2001, pp. 1587-1596.

[3] X. Meng, V. Pappas and L. Zhang, "Improving the scalability of data center networks with traffic-aware virtual machine placement," in *Proc. IEEE INFOCOM*, 2010, pp. 1-9.

[4] A. Beloglazov, J. Abawajy and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755-768, 2012.

[5] S. L. Hakimi, "Optimum locations of switching centers and the absolute centers and medians of a graph," *Operations Research*, vol. 12, no. 3, pp. 450-459, 1964.

[6] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48-50, 1956.

[7] R. C. Prim, "Shortest connection networks and some generalizations," *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389-1401, 1957.

[8] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269-271, 1959.

[9] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.

[10] S. Warshall, "A theorem on Boolean matrices," *Journal of the ACM*, vol. 9, no. 1, pp. 11-12, 1962.

[11] M. Charikar, S. Guha, E. Tardos and D. B. Shmoys, "A constant-factor approximation algorithm for the k-median problem," *Journal of Computer and System Sciences*, vol. 65, no. 1, pp. 129-149, 2002.

[12] V. Arya, N. Garg, R. Khandekar, A. Meyerson, K. Munagala and V. Pandit, "Local search heuristics for k-median and facility location problems," *SIAM Journal on Computing*, vol. 33, no. 3, pp. 544-562, 2004.

[13] O. Kariv and S. L. Hakimi, "An algorithmic approach to network location problems. II: The p-medians," *SIAM Journal on Applied Mathematics*, vol. 37, no. 3, pp. 539-560, 1979.

[14] A. Tamir, "An $O(pn^2)$ algorithm for the p-median and related problems on tree graphs," *Operations Research Letters*, vol. 19, no. 2, pp. 59-64, 1996.

[15] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton University Press, 1957.

[16] R. K. Ahuja, T. L. Magnanti and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

APPENDIX – I

Screen shots of work done.

[Insert screenshots of the code running, the result tables and the generated figures here.]

Screen shots of deployment tools of

- Hacker-rank
- Leet codes
- GitHub

[Insert screenshots here.]

APPENDIX - II

PSEUDOCODE AND REPRODUCIBILITY

This appendix gives the pseudocode of the placement pipeline and the information needed to reproduce the experiments in Chapter 4.

A.1 Pseudocode of the Placement Pipeline

```
INPUT: graph G=(V,E,w), demand d[i], capacity C, replica budget K

1. mst  <- Kruskal(G)                        # union-find, O(E log E)
2. full <- Dijkstra from every node on G     # shortest-path latencies
   tree <- Dijkstra from every node on mst   # backbone latencies
3. S    <- TreeDP(mst, tree, d, K)           # exact k-median on the tree
4. x    <- MinCostFlow(full or tree, d, S, C)
5. return S, x, sum(dist[i][s] * x[i][s]) / sum(d)
```

A.2 Pseudocode of the Tree DP

```
TreeDP(tree T rooted at r, dist, d, K):
  for v in post-order:
      P[j][u]      <- min-plus merge over children c of X_c[., u]
      Popen[j]     <- min-plus merge over children c of X_c[., v]
      f[v][j][u]   <- d[v] * dist[v][u] + P[j][u]          (u != v)
      f[v][1+j][v] <- Popen[j]                             (v is open)
      m[v][j]      <- min over z in subtree(v) of f[v][j][z]
      X_v[j][u]    <- f[v][j][u]                  if u in subtree(v)
                      min(f[v][j][u], m[v][j])    otherwise
  cost(K) <- m[r][K]
  recover the replica set by backtracking the stored merge indices
```

A.3 Reproducing the Experiments

The experiments use seeds 0 to 29 for the 50-server study and seeds 500 to 519 for the 14-server study. Topology, demand and random-baseline draws are all derived from these seeds, so the numbers in Chapter 4 can be regenerated exactly.

[Add the repository link or attach the source code listing here.]